

One-to-One (1:1) Database Relationship

One-to-One relationship ka matlab hai:

Table A ka **ek record** Table B ke **sirf ek record** se related hota hai, aur Table B ka bhi ek record Table A ke **sirf ek record** se related hota hai.

1. Real-life example

Person ↔ Passport

- Ek person → ek passport
- Ek passport → ek person

Person 1 ————— 1 Passport



2. Tables ka structure

Person Table

person_id (PK)	name	age
1	Tarun	22
2	Rahul	23
3	Amit	21

Passport Table

passport_id (PK)	passport_number	person_id (FK)	
101	P12345	1	
102	P67890	2	
103	P55555	3	

3. One-to-One ki zarurat kyu padi?

Problem	One-to-One solution
Ek table bahut bada ho raha hai	Data ko separate tables me divide kar sakte hain
Sensitive data alag rakhna hai	Separate table bana sakte hain
Optional information hai	Dusri table me rakh sakte hain
Data ko logically separate rakhna hai	Related data ko separate entities me divide karte hain
Security/access control chahiye	Sensitive table par restricted access de sakte hain

Example

User ki normal information:

User

user_id	name	email
1	Tarun	tarun@gmail.com ↗

Sensitive/profile information:

UserProfile

profile_id	user_id (FK)	address	Aadhaar
101	1	Indore	XXXXX

Isse **User** table unnecessarily huge nahi hoti.

4. Important condition

One-to-One enforce karne ke liye FK ko **UNIQUE** karna important hai.

`</>` SQL

```
CREATE TABLE UserProfile (
  profile_id INT PRIMARY KEY,
  user_id INT UNIQUE,
  address VARCHAR(100),

  FOREIGN KEY (user_id)
  REFERENCES Users(user_id)
);
```

UNIQUE ki wajah se:

`user_id = 1` → sirf ek profile

Ye allowed nahi hoga:

Ye allowed nahi hoga:

profile_id	user_id
101	1
102	1 ✗

Kyunki `user_id = 1` duplicate ho raha hai.

5. One-to-One vs One-to-Many

Relationship	Example	Meaning	
1:1	Person → Passport	1 person = 1 passport	
1:N	Customer → Orders	1 customer = many orders	
N:N	Students ↔ Courses	Many students = many courses	



One-to-Many (1:N) Database Relationship

One-to-Many ka simple meaning:

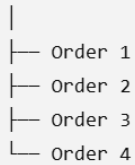
Ek record ke saath doosri table ke multiple records related ho sakte hain, lekin doosri table ka har record normally sirf ek parent record se related hota hai.

Real-life example

Customer → Orders

Ek customer **multiple orders** kar sakta hai:

Customer



customers

customer_id (PK)	name	email
1	Tarun	tarun@gmail.com ↗
2	Rahul	rahul@gmail.com ↗
3	Amit	amit@gmail.com ↗

orders

order_id (PK)	product	amount	customer_id (FK)	
101	Laptop	50000	1	
102	Mouse	500	1	
103	Keyboard	1500	1	
104	Mobile	25000	2	
105	Headphone	2000	3	



Notice:

```
Customer 1 (Tarun)
  |
  ├── Order 101
  ├── Order 102
  └── Order 103
```

```
Customer 2 (Rahul)
  |
  └── Order 104
```

```
Customer 3 (Amit)
  |
  └── Order 105
```

Yahi **1:N relationship** hai.

8b8a02f41a9e

2. Relationship kaise create hota hai?

Parent table:

 SQL

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Child table:

 SQL

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    product VARCHAR(100),  
    amount DECIMAL(10,2),  
    customer_id INT,  
  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)  
);
```



3. Data Insert

Customers

</> SQL

```
INSERT INTO customers
(customer_id, name, email)
VALUES
(1, 'Tarun', 'tarun@gmail.com'),
(2, 'Rahul', 'rahul@gmail.com'),
(3, 'Amit', 'amit@gmail.com');
```

Orders

</> SQL

```
INSERT INTO orders
(order_id, product, amount, customer_id)
VALUES
(101, 'Laptop', 50000, 1),
(102, 'Mouse', 500, 1),
(103, 'Keyboard', 1500, 1),
(104, 'Mobile', 25000, 2),
```



Yahan:

```
customer_id = 1
```

multiple times aa sakta hai:

```
101 → 1
102 → 1
103 → 1
```

Kyunki ek customer ke multiple orders ho sakte hain.

4. JOIN karke data kaise nikale?

`</>` SQL

SELECT

```
customers.name,  
orders.order_id,  
orders.product,  
orders.amount
```

FROM customers

JOIN orders

ON customers.customer_id = orders.customer_id;

Output:

name	order_id	product	amount	
Tarun	101	Laptop	50000	
Tarun	102	Mouse	500	
Tarun	103	Keyboard	1500	
Rahul	104	Mobile	25000	
Amit	105	Headphone	2000	

5. Ek customer ke saare orders

Agar Tarun ke orders chahiye:

`</> SQL`

```
SELECT *
FROM orders
WHERE customer_id = 1;
```



Output:

order_id	product	amount	customer_id
101	Laptop	50000	1
102	Mouse	500	1
103	Keyboard	1500	1

6. One-to-Many me UNIQUE kyu nahi lagate?

One-to-One me:

`</> SQL`

```
user_id INT UNIQUE
```

lagate hain.

Lekin One-to-Many me **UNIQUE nahi lagana** chahiye.

✗ Wrong:

`</> SQL`

```
customer_id INT UNIQUE
```

Aisa karne par customer **1** ka sirf ek order ho payega.

✓ Correct:

SQL

```
customer_id INT
```

Ab:

```
customer_id = 1  
customer_id = 1  
customer_id = 1
```

allowed hai.

7. Foreign Key ka main purpose

Foreign key ensure karti hai ki order kisi **existing customer** ka hi ho.

Suppose:

`</>` SQL

```
INSERT INTO orders  
VALUES (106, 'Monitor', 10000, 99);
```

Agar customer **99** exist nahi karta, to foreign-key constraint error aa sakta hai.

Matlab:

```
Customer 99 ❌ doesn't exist  
↓  
Order 106 ❌ cannot reference it
```

Isse **referential integrity** maintain hoti hai.

8. ON DELETE CASCADE

Agar customer delete hone par uske saare orders bhi delete karne hain:

`</> SQL`

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  product VARCHAR(100),  
  amount DECIMAL(10,2),  
  customer_id INT,  
  
  FOREIGN KEY (customer_id)  
  REFERENCES customers(customer_id)  
  ON DELETE CASCADE  
);
```

Ab:

`</> SQL`

```
DELETE FROM customers  
WHERE customer_id = 1;
```

To customer 1 ke orders bhi delete ho jayenge:

Customer 1



Order 101 ✖
Order 102 ✖
Order 103 ✖

9. Different real-world examples

Parent (1)	Child (Many)
Customer	Orders
Department	Employees
Teacher	Students
User	Posts
User	Comments
Country	States
Company	Employees
Category	Products
Author	Books
Order	Order Items

Department 1

- |
- |— Employee 1
- |— Employee 2
- |— Employee 3
- |— Employee 4



10. Database design rule ★

One-to-Many relationship me Foreign Key hamesha "Many" side wali table me hoti hai.

Customers		Orders
-----		-----
customer_id PK	←	customer_id FK
1		101
2		102
3		103



Yaad rakhne ka easiest rule:

"Foreign Key goes to the Many side."



One-to-One vs One-to-Many

Feature	1:1	1:N
Example	User → Profile	Customer → Orders
One parent	One child	Many children
FK	Child table	Many-side table
FK UNIQUE?	Usually yes	✗ No
Example	1 User → 1 Profile	1 Customer → 10 Orders

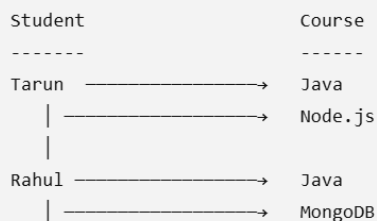
Many-to-Many (N:N) Database Relationship

Many-to-Many ka matlab:

Table A ka ek record, Table B ke **multiple records** se related ho sakta hai, aur Table B ka ek record bhi Table A ke **multiple records** se related ho sakta hai.

Real-life example: Students ↔ Courses

Ek student multiple courses kar sakta hai, aur ek course ko multiple students kar sakte hain.



So:

Students N — N Courses

00a02141a5c

1. Problem kya thi?

Agar hum sirf 2 tables banaye:

Students

student_id	name
1	Tarun
2	Rahul

Courses

course_id	course_name
101	Java
102	Node.js
103	MongoDB

2. Junction / Bridge / Mapping Table

Students

|
|



Student_Course



|
|

Courses

students

student_id (PK)	name
1	Tarun
2	Rahul
3	Amit

courses

course_id (PK)	course_name
101	Java
102	Node.js
103	MongoDB

student_course

student_id (FK)	course_id (FK)
1	101
1	102
1	103
2	101
2	103
3	102

Ab clearly:

Tarun → Java

Tarun → Node.js

Tarun → MongoDB

Rahul → Java

Rahul → MongoDB

Amit → Node.js

Students table

</> SQL

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100)  
);
```

Courses table

</> SQL

```
CREATE TABLE courses (  
  course_id INT PRIMARY KEY,  
  course_name VARCHAR(100)  
);
```

Junction table

</> SQL

```
CREATE TABLE student_course (
  student_id INT,
  course_id INT,

  PRIMARY KEY (student_id, course_id),

  FOREIGN KEY (student_id)
    REFERENCES students(student_id),

  FOREIGN KEY (course_id)
    REFERENCES courses(course_id)
);
```

Composite Primary Key hai.

Iska matlab same student ko same course me duplicate entry nahi de sakte.



student_id	course_id
1	101
1	101

Duplicate relationship.

4. Data Insert

Students

</> SQL

```
INSERT INTO students
VALUES
(1, 'Tarun'),
(2, 'Rahul'),
(3, 'Amit');
```

Courses

</> SQL

```
INSERT INTO courses
VALUES
(101, 'Java'),
(102, 'Node.js'),
(103, 'MongoDB');
```

Relationships

SQL

```
INSERT INTO student_course
VALUES
(1, 101),
(1, 102),
(1, 103),
(2, 101),
(2, 103),
(3, 102);
```

5. JOIN — Student ke courses

Tarun ne kaunse courses liye?

SQL

```
SELECT
    s.name,
    c.course_name
FROM students s
JOIN student_course sc
    ON s.student_id = sc.student_id
JOIN courses c
    ON sc.course_id = c.course_id
WHERE s.student_id = 1;
```

Output:

name	course_name
Tarun	Java
Tarun	Node.js
Tarun	MongoDB

6. Course ke students

Java course me kaun-kaun hai?

</> SQL

```

SELECT
    c.course_name,
    s.name
FROM courses c
JOIN student_course sc
    ON c.course_id = sc.course_id
JOIN students s
    ON sc.student_id = s.student_id
WHERE c.course_id = 101;

```

Output:

course_name	name
Java	Tarun
Java	Rahul

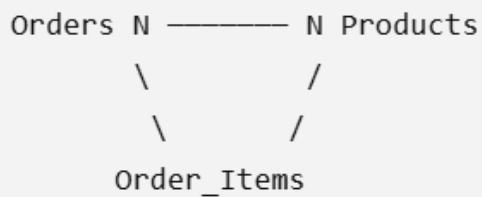
7. Ye third table kyu zaroori hai?

Without Junction Table	With Junction Table
Relationship properly store nahi hota	Relationship clearly store hota hai
Duplicate/repeating data aa sakta hai	Normalized design
Queries difficult hoti hain	JOIN se easy
Student → multiple courses difficult	Easily supported
Course → multiple students difficult	Easily supported

Products ↔ Orders

Ek order me multiple products ho sakte hain.

Aur ek product multiple orders me aa sakta hai.



orders

order_id	customer
1	Tarun
2	Rahul

products

product_id	product
101	Laptop
102	Mouse
103	Keyboard

order_items

order_id	product_id	quantity
1	101	1
1	102	2
2	101	1
2	103	1

Yahan **quantity** relationship ki additional information hai.

Isi wajah se junction table bahut useful hoti hai.

9. One-to-One vs One-to-Many vs Many-to-Many

Relationship	Example	Foreign Key
1:1	User → Profile	Child table
1:N	Customer → Orders	Many-side table
N:N	Students ↔ Courses	Junction table

Sabse important rule ★

1 : 1

Parent — Child

1 : N

Parent —< Children

↑

FK

N : N

Table A — Junction Table — Table B

↑

FK

↑

FK

Entity-Relationship (ER) kya hai?

Entity-Relationship (ER) database design ka ek **model/diagram** hai jisse hum database me:

- kaun-kaun si **entities/tables** hongi
- unke andar kaunse **attributes/columns** honge
- tables ke beech kya **relationship** hai

ye design karte hain.

Simple language me:

ER model database banane se pehle uska blueprint hota hai.

1. Entity kya hoti hai?

Entity = real-world object jiska data database me store karna hai.

Real-world object	Entity/Table
Student	Student
Teacher	Teacher
Customer	Customer
Product	Product
Order	Order

2. Attribute kya hota hai?

Entity ki properties ko **attributes** kehte hain.

Example:

```
Student
|
|— student_id
|— name
|— email
|— age
```

Entity	Attributes
Student	student_id, name, email, age
Product	product_id, name, price
Customer	customer_id, name, email



Database me attributes generally **columns** ban jaate hain.

3. Relationship kya hota hai?

Do entities ke beech connection ko **relationship** kehte hain.

Example:

```
Student — Enrolls — Course
```

Meaning:

Student course me enroll karta hai.

4. Cardinality

Relationship me ye batata hai ki **kitne records ek doosre se related ho sakte hain.**

Cardinality	Meaning	Example
1:1	One-to-One	User ↔ Profile
1:N	One-to-Many	Customer → Orders
N:N	Many-to-Many	Student ↔ Courses

7. ER Model vs Database Table

ER Model	Database
Entity	Table
Attribute	Column
Entity instance	Row
Relationship	Foreign Key / Junction table
Primary attribute	Primary Key

8. ER Diagram kyu zaroori hai?

Database directly code karne se pehle ER model banane se:

Benefit	Kya help milti hai
Planning	Database structure pehle clear hota hai
Relationships	1:1, 1:N, N:N identify kar sakte hain
Normalization	Duplicate data reduce kar sakte hain
Communication	Developer/team ko design samajh aata hai
Error reduction	Galat relationships early identify hote hain
Scalability	Future tables/relationships plan karna easy hota hai